

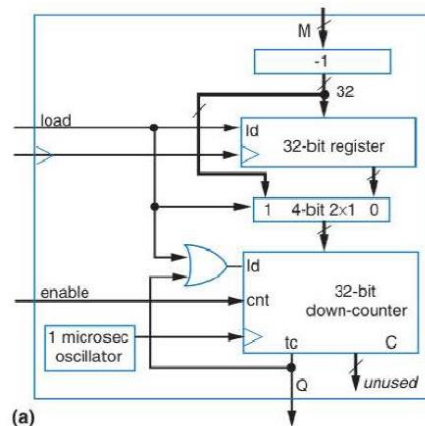
Examen SN361**3^{ème} année 1^{ère} session 2024-2025****Durée : 120 mn****Document autorisé : syntaxe VHDL essentielle****Calculatrice interdite**

Les points donnés dans l'énoncé entre crochets [X] après chaque question représentent le barème et indiquent le temps à passer en minutes sur chaque question.

Exercice 1 : Conception et validation d'un « timer » [90]

L'objectif de cet exercice est de concevoir et valider un timer, c'est-à-dire un circuit qui peut être programmé pour générer périodiquement une impulsion courte (d'une période d'horloge de large) avec une période de temps spécifié (par exemple, une impulsion courte toutes les 300 ms).

Ce timer est contrôlé par une horloge de période 1 μ s qui fournit la référence temporelle. Ainsi, si on souhaite une impulsion toutes les 50 μ s, il faut programmer le timer avec la valeur 50. Le timer a une largeur de 32 bits. La figure ci-dessous représente la conception interne du timer. Le « bloc -1 » représente une opération de décrémentation de -1.



1 Quelle valeur doit être programmée (c'est-à-dire envoyée vers M) pour générer une impulsion toutes les 300 ms ? [4]

2 Afin d'expliquer le fonctionnement de ce timer, faire un chronogramme représentant les signaux externes (M, load, clk, enable, Q) et internes (M32 en sortie du « bloc -1 », R32 en sortie du registre de 32 bits, Mux32 en sortie du multiplexeur, OR1 en sortie de la porte OU et C) **pour deux impulsions avec M=3 puis pour deux impulsions avec M=2 [20].**

Remarques :

- Sur la figure, l'entrée clk est connectée au port externe avec un triangle : vous considérerez **que cette horloge clk est parfaitement identique à l'horloge interne** générée par l'oscillateur.
- tc=1 quand C atteint 0.
- Tous les signaux sont synchrones avec l'horloge clk

3 Est-ce que les périodes des impulsions générées sur vos chronogrammes correspondent aux valeurs de M successives (3 puis 2) ? Si ce n'est pas le cas, expliquez [4].

4 Donnez une description en VHDL de l'entité du timer [4].

Remarques :

- Utilisez uniquement des signaux de type *std_logic* ou *std_logic_vector*
- Déclarez les bibliothèques et paquetages nécessaires aux types et aux opérations arithmétiques ensuite utilisées dans l'architecture

5 Décrire en VHDL RTL l'architecture *rtl* de ce circuit. Pour cela vous utiliserez les processus et instructions concurrentes suivantes :

- un processus pour le registre R32 [10],

- un processus explicite pour le décompteur [15],
- une instruction concurrente pour le signal M32 [5],
- une instruction concurrente pour le multiplexeur Mux32 [5].

6 Commentez la fonction du code verilog ci-dessous : [5]

```
always @(load or M32 or R32)
```

```
begin
```

```
case (load)
```

```
    1'b0: MUX32 = R32;
```

```
    1'b1: MUX32 = M32;
```

```
endcase
```

```
end
```

6 Combien de bascules générera la synthèse de votre architecture précédente ? Justifiez [4].

7 Donnez une description du testbench *tb_timer* permettant de retrouver **exactement le même chronogramme que celui que vous avez donné à la question 2** [15]

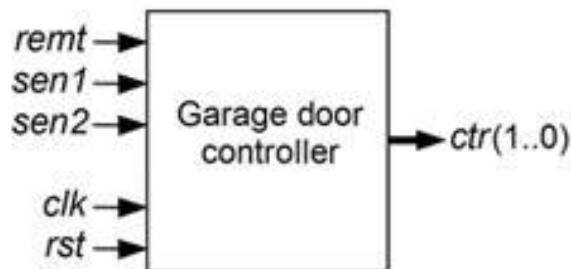
Exercice 2 : Contrôleur de porte de garage [30]

Cet exercice vise à dessiner le diagramme de transitions d'un contrôleur de porte de garage qui fonctionne comme suit.

Si la porte est complètement fermée ou complètement ouverte et que la télécommande est activée, le moteur est mis en marche dans le sens de l'ouverture ou de la fermeture, respectivement.

Si la porte s'ouvre ou se ferme et que la télécommande est activée, la porte s'arrête. Si la télécommande est à nouveau activée, le moteur est activé pour déplacer la porte dans la direction opposée.

Les ports du circuit sont représentés sur la figure ci-dessous, avec *remt* (commande de la télécommande), *sen1* (détection de porte ouverte), *sen2* (détection de porte fermée) en entrée (plus les signaux conventionnels *clk* et *rst*) et *ctr* (contrôle) est la sortie.



Noter que *ctr* a deux bits : *ctr(1)* à '1' le moteur est ON, *ctr(1)* à '0' le moteur est OFF, *ctr(0)* définit le sens de rotation du moteur : *ctr(0)*='0' pour l'ouverture, *ctr(0)*='1' pour la fermeture.

Quand le moteur est OFF la valeur de *ctr(0)* importe peu.

Quand *rst* est activé alors le contrôleur se place dans l'état où la porte est considérée fermée.

On considère que **le signal *remt* est constitué d'impulsion à 1 d'une seule période de large** (à chaque fois que l'on appuie sur la télécommande une seule impulsion à 1 est générée) et que toutes les entrées sont parfaitement synchronisées.

2.1 Donnez un diagramme de transitions de la machine à états de Moore de ce contrôleur [15]

Maintenant à chaque fois que l'on reste appuyé sur la télécommande on génère un signal *remt* à 1 (pendant toute la durée pendant laquelle on reste appuyé). Il faut alors détecter les passages à 0 pour contrôler correctement le moteur.

2.2 Donnez un diagramme de transitions de la machine à états de Moore de ce nouveau contrôleur [15]